

HW 3 – Multimodal Vision-Language Models

MMAI 2026 – Multimodal Machine Learning

Peiran Niu

March 2026

Part 1: Reading & Reflection (20 pts)

Q1 – Multimodal Data Noise and Cleaning

Typical noise in web-crawled multimodal datasets includes misaligned pairs (e.g., an image of a dog with alt-text 'IMG_1234.jpg'), low-quality or blurry visuals, and text containing irrelevant SEO keywords or toxic content. This noise forces the model to learn incorrect associations, diluting performance and robustness. The 'Quality Not Quantity' paper shows that simply throwing more noisy data at a model can degrade the robust features learned from a smaller, higher-quality dataset. A noisy data point could be a detailed architectural diagram paired with a generic caption like 'building', or a cat photo where the text is entirely unrelated spam.

We can build estimators using contrastive models like CLIP: computing cosine similarity between image and text embeddings filters out pairs below a quality threshold — the approach used when curating the LAION dataset. With unlimited funding, the ideal strategy is a massive human-in-the-loop pipeline: models flag suspect pairs, and human experts verify alignment, filter biases, and rewrite sparse alt-text into dense, descriptive captions — achieving both scale and precision.

Q2 – Frozen LLMs as Multimodal Backbones

The core intuition of using frozen LLMs as multimodal backbones is that LLMs already possess encyclopedic knowledge, reasoning capabilities, and few-shot skills from pre-training on vast text corpora. Freezing preserves this powerful foundation without catastrophic forgetting and saves enormous compute. To bridge modalities, vision encoders (ResNets or Vision Transformers) are trained to map images into continuous embeddings compatible with the LLM's input space.

The LLM then processes visual embeddings as a 'visual prefix': the encoder translates the image into the same embedding space as text tokens, and the frozen LLM reads this sequence of visual tokens as contextual input before auto-regressively generating the text response — no architectural change needed.

Q3 – Selecting Impactful Instruction-Tuning Data

Impactful instruction-tuning data must score high on complexity, diversity, and coverage — not just volume. Thousands of trivial 'What color is the car?' pairs are redundant; valuable data forces spatial reasoning, OCR, or multi-hop logic. One strategy is using a strong 'teacher' model (e.g., GPT-4V) to score difficulty and quality of candidate pairs, preferring those requiring non-trivial reasoning.

Another strategy is core-set selection in embedding space: clustering similar examples and sampling only a few high-quality representatives per cluster. This produces a balanced, wide task distribution without redundancy, ensuring fine-tuning data is both efficient and effective for the target task.

Q4 – Distinguishing AI vs. Human-Created Content

Beyond watermarking, current AI-content detection exploits residual artifacts. For text, statistical analysis of perplexity and 'burstiness' exposes AI writing: models produce more predictable token sequences than humans. For images and video, algorithms look for physical inconsistencies (asymmetrical pupils, unnatural lighting, morphing backgrounds) or analyse high-frequency spectral noise at the pixel level — real camera sensors leave characteristic sensor patterns that synthetic images lack.

However, this is becoming an arms race: as generative models improve, such artifacts shrink. Without cryptographic provenance tracking baked into hardware and software (e.g., the C2PA standard), post-hoc analysis alone is likely to become insufficient — reliably distinguishing AI from human content across all modalities may soon be practically impossible without provenance infrastructure.

Q5 – Sora, Physics, and the Path to AGI

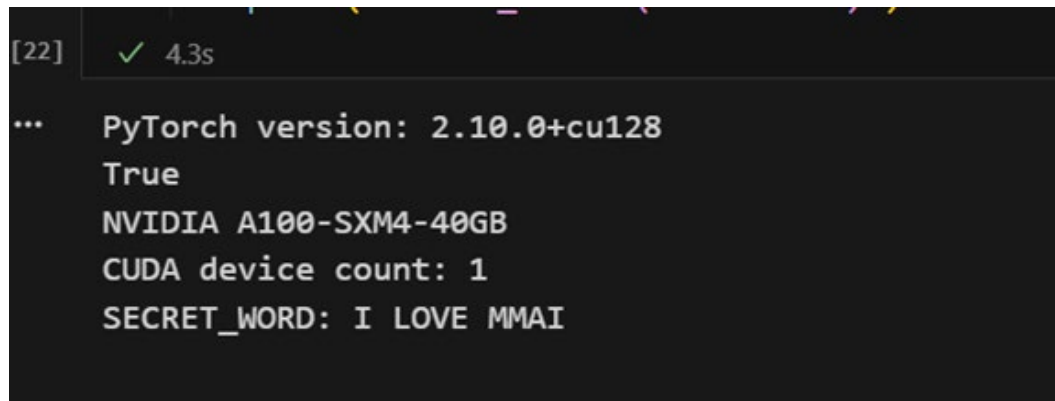
LeCun makes a fair point: Sora-class models predict pixel or patch probabilities from massive training data rather than explicitly simulating 3D geometry or physics — explaining why videos occasionally show objects floating, merging, or defying gravity. The model matches patterns rather than reasoning over physical laws.

For generative AI to incorporate genuine real-world knowledge, future systems will likely need neuro-symbolic hybrids that combine neural perception with explicit physics engines or logic solvers to constrain generation within physically plausible boundaries. While generative AI is a massive leap for machine perception and language, passive pattern-matching alone is probably insufficient for AGI: true AGI requires causal reasoning, long-horizon planning, and active environment exploration — not just static training data imitation.

Part 2: Testing and Fine-tuning VLMs (100 pts)

Problem 1: GPU Verification and Library Installation

Environment verified on an NVIDIA A100-SXM4-40GB (Colab Pro). PyTorch 2.10.0+cu128 detected CUDA successfully (`torch.cuda.is_available() = True`, device count = 1). The GPU-based XOR cipher decoded the secret word: “I LOVE MMAI” — confirming that both CUDA tensors and GPU synchronisation work correctly before any model loading.

A terminal window with a dark background. The prompt is [22]. To its right is a green checkmark and the text 4.3s. The output consists of several lines of text: PyTorch version: 2.10.0+cu128, True, NVIDIA A100-SXM4-40GB, CUDA device count: 1, and SECRET_WORD: I LOVE MMAI.

```
[22] ✓ 4.3s
... PyTorch version: 2.10.0+cu128
    True
    NVIDIA A100-SXM4-40GB
    CUDA device count: 1
    SECRET_WORD: I LOVE MMAI
```

Figure 1.1: GPU verification output (PyTorch + CUDA checks)

Problem 2: Prepare Your Dataset (20 pts)

2.1 Data Source and Format

The dataset was prepared from the provided HW3 data package, which contains four OR benchmark categories: NLP4LP (264 instances), LogiOR (92), IndustryOR (83), and ComplexOR (18) — 457 instances in total. For each instance, a MILP bipartite-graph JSON was constructed from the raw MPS file (or loaded from pre-built JSON files), and a heatmap image was rendered from the constraint matrix. Each (image, question, answer) triple was then written to `data.jsonl` following the `mmai-data` format.

2.2 Graph JSON Construction

Each MILP instance was converted to a bipartite graph JSON with three sections: variable nodes (storing type B/I/C, bounds, objective coefficient), constraint nodes (storing sense $\leq/\geq/=$, RHS, plus an explicit objective node), and edges (non-zero constraint matrix entries keyed by constraint and variable name). This graph JSON is the canonical intermediate representation from which both the heatmap image and the structured ground-truth answer are derived, ensuring consistency between the visual input and the target text.

2.3 Heatmap Image Generation

Heatmaps were rendered as raster PNG images (288×288 px minimum). The main body encodes non-zero coefficients as a diverging colormap (blue = negative, white = zero, red = positive). A top colour band encodes variable types (B/I/C) and a left band encodes constraint

senses. This layout gives the VLM all structural information needed to answer the target question in a single image without access to the raw numbers.

2.4 Dataset Visualizations

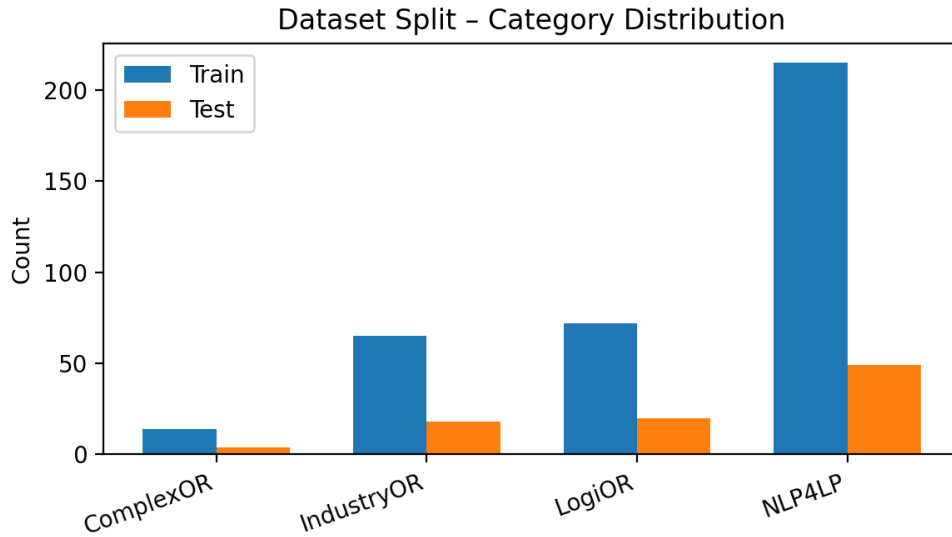


Figure 2.1: Train / Test Split by Domain Category

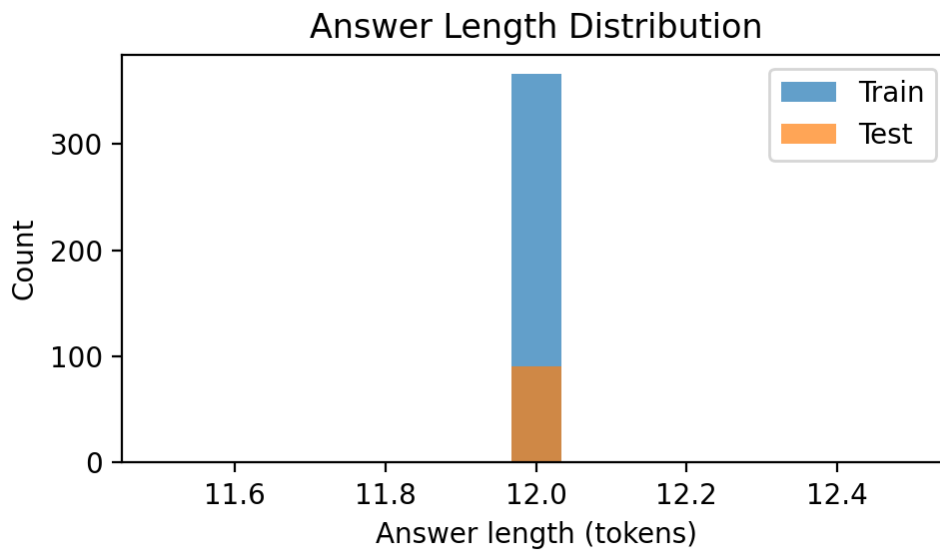


Figure 2.2: Answer Token-Length Distribution (Train vs Test)

2.5 Questions: Data Conversion and Split Strategy

Issues with Converting Non-Image Data to Images

Issues with converting non-image data into images:

1. Precision loss — floating-point coefficients are mapped to a discrete colour scale, so small

numerical differences become invisible (0.001 vs 0.0001 may render identically).

2. Ordering ambiguity — heatmap appearance depends entirely on the (arbitrary) ordering of rows (constraints) and columns (variables); two identical MILP instances can look radically different under different permutations.

3. Scale sensitivity — a single large outlier coefficient dominates the colour range and hides the structure in all remaining cells.

4. Categorical encoding — constraint sense ($\leq/\geq/=$) and variable type (B/I/C) must be represented as auxiliary colour bands rather than natural visual features; the model must learn this convention explicitly.

Drawbacks of Visual Representations

Drawbacks of using visual (heatmap) representations:

Even with careful design, heatmaps have inherent limits for MILP reasoning. The model must count exact pixel columns/rows to determine variable or constraint counts — unlike text, where integers are explicit tokens. Our baseline results confirm this: the model outputs 35 variables for 2-variable instances, suggesting it guesses a ‘typical’ size rather than reading the actual column count. Any attribute requiring sub-pixel precision (e.g., density to 3 decimal places, exact coefficient values) is poorly conveyed by a raster image. By contrast, embedding the constraint matrix as JSON text would preserve exact values at the cost of losing the spatial/structural overview that images provide naturally.

Train / Test Split Strategy

Split strategy — stratified 90 % train / 10 % test:

The data was split stratified by domain category (NLP4LP, LogiOR, IndustryOR, ComplexOR) so each problem type is proportionally represented in both train and test. This ensures the model encounters all four domains during training and is evaluated on held-out instances of each type — avoiding a test set dominated by NLP4LP (the largest category at ~264 examples). An alternative random 80/20 split without stratification was considered but rejected because it risked skewing the test distribution. A 90/10 ratio (rather than 80/20) was chosen because the dataset is small (~457 instances); maximising training data is more important than a larger test set at this scale.

Problem 3: Baseline Inference (10 pts)

The pre-trained Qwen2.5-VL-3B-Instruct model was evaluated on 30 held-out test images. Although the homework handout demonstrates baseline inference on 4 sample images, I used 30 images for a more reliable estimate. Each image was paired with the same structured question asking for Variables, Constraints, Density, Binary %, Integer %, and Objective direction.

Image	Ground Truth	Model Output
test_NLP4LP_prob_257	Vars: 2, Cons: 2, Density: high, Bin: 0%, Int: 100%, Obj: minimize	4, 6, 0.5, 20%, 80%, minimize
test_IndustryOR_prob_057	Vars: 8, Cons: 9, Density: medium, Bin: 50%, Int: 0%, Obj: maximize	4, 6, 0.5, 20%, 80%, maximize
test_NLP4LP_prob_172	Vars: 2, Cons: 2, Density: high, Bin: 0%, Int: 100%, Obj: minimize	4, 3, 0.5, 20%, 10%, minimize
test_IndustryOR_prob_022	Vars: 3, Cons: 2, Density: high, Bin: 0%, Int: 100%, Obj: maximize	4, 3, 0.5, 20%, 10%, minimize

What worked: The model produced parseable output and occasionally matched the objective direction (50 % accuracy with Variant 1, though largely by defaulting to ‘minimize’).

What failed: The model struggled severely with exact counting and domain-specific visual encodings. It hallucinated variable and constraint counts (e.g., predicting 4–6 for a 2-variable instance), scoring 20 % and 0 % respectively on Variant 1. It also failed to interpret the top color band for variable types (Binary 6.7 %, Integer 3.3 %). Density was always wrong (0 %). Overall, the base VLM can follow an output format but lacks the zero-shot ability to parse the spatial and color-encoded semantics of MILP heatmaps.

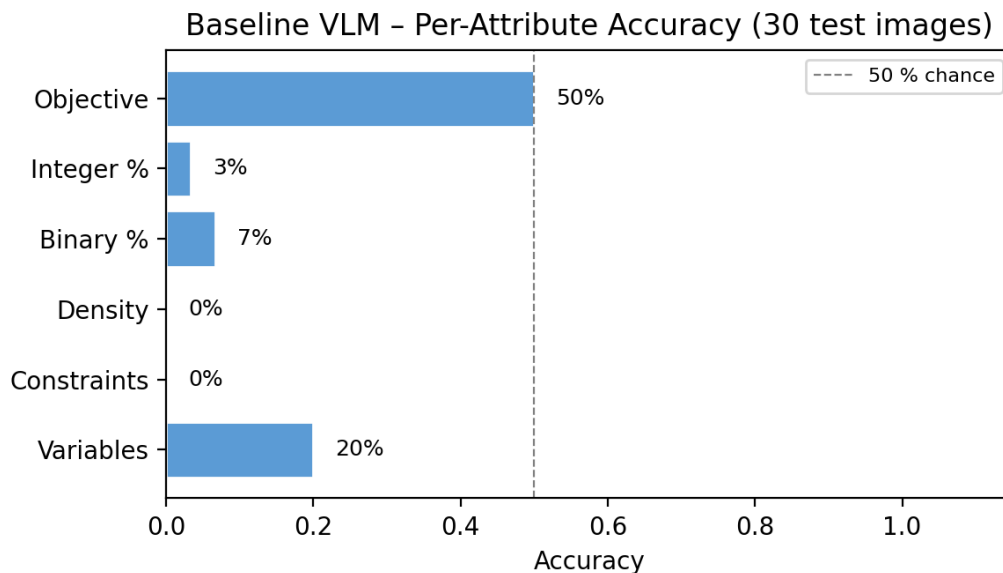


Figure 3.1: Baseline VLM Per-Attribute Accuracy on 30 Test Images

Problem 4: Prompt Engineering (15 pts)

I tested four SYSTEM_PROMPT variants on the same 30 held-out test images, and all four produced identical per-attribute accuracy curves (Figures 4.1–4.4).

Variant 1 (Strict format + expert role) mostly controlled formatting, but it did not improve the model’s ability to read the heatmap. Variant 2 (Few-shot with an example answer) also did not change accuracy: the model could imitate the output template, yet still hallucinated counts and percentages. Variant 3 (Step-by-step reasoning) often generated longer, seemingly reasoned explanations (e.g., “count columns/rows”), but the underlying predictions were not more correct; the model frequently reported implausible sizes (6, 10, 35, etc.) that do not match the ground truth. Variant 4 (Minimal, no fluff) forced short outputs, but errors remained the same.

Interpretation: prompt engineering changed the *style* of responses more than the *content*. The dominant failure mode appears to be visual perception: the key signals (row/column counts, subtle sparsity/density, and thin color bands for variable types) are hard to decode reliably from a low-resolution raster heatmap. When the visual evidence is not extracted correctly, additional instructions (examples, step-by-step, or stricter formatting) cannot rescue accuracy. This motivates fine-tuning (Problem 5) and/or using higher-resolution images or a more explicit structured representation for attributes like density.

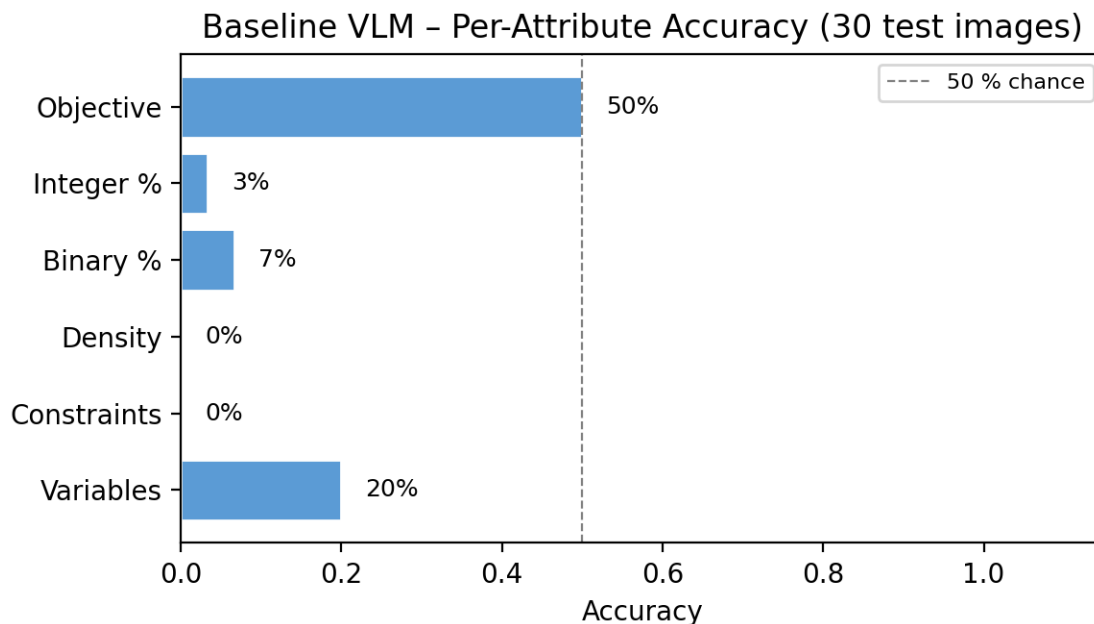


Figure 4.1: Prompt Variant 1 (Strict format + expert role) — per-attribute accuracy on 30 test images

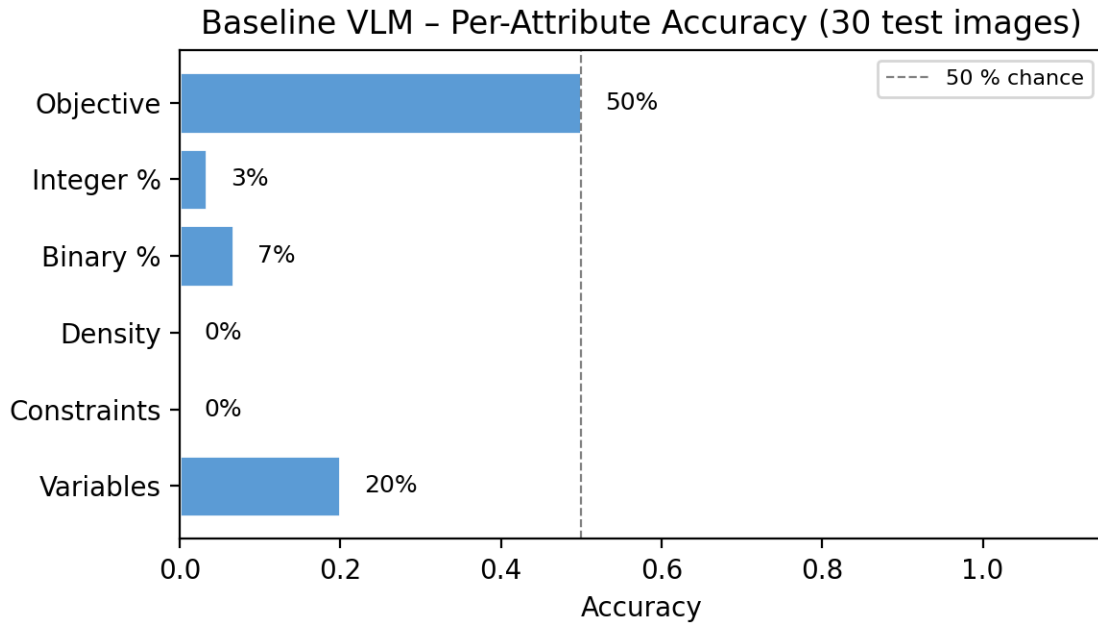


Figure 4.2: Prompt Variant 2 (Few-shot with example answer) — per-attribute accuracy on 30 test images

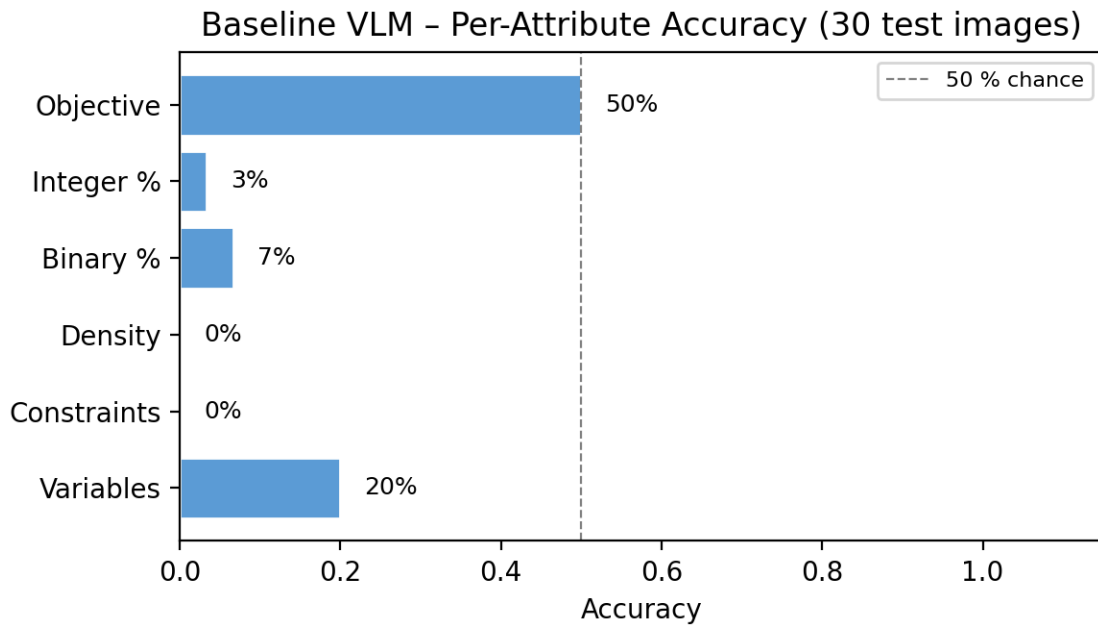


Figure 4.3: Prompt Variant 3 (Step-by-step reasoning) — per-attribute accuracy on 30 test images

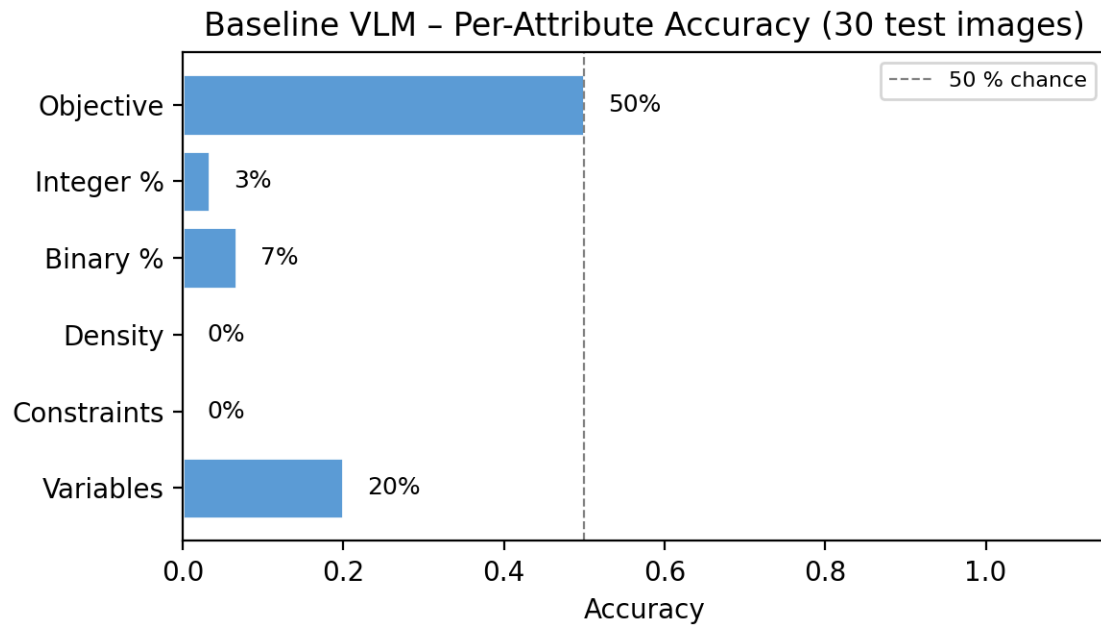


Figure 4.4: Prompt Variant 4 (Minimal, no fluff) — per-attribute accuracy on 30 test images

Problem 5: LoRA Fine-Tuning (20 pts)

5.1 Best-Performing Settings

Best-performing LoRA configuration:

Epochs: 5 | Learning rate: $1e-4$ | Batch size: 1 | Grad accum: 1
Max seq len: 512 | Image shortest edge: 288 px | Eval split: 10 %
LoRA rank (r): 4 | LoRA alpha: 8 | Dropout: 0.05
Target modules: q_proj, k_proj, v_proj, o_proj

Trainable parameters: < 0.1 % of total model weights. Training was run with max_steps=300 and early stopping (patience=2 on eval_loss) on an NVIDIA A100 (FP16).

5.2 Hyperparameter Analysis

Most impactful hyperparameters:

1. Number of epochs / steps — Because the dataset is small (~360 examples), the model required sufficient iterations (up to 500 steps) to update the attention heads to map color bands to categorical values.
2. Image Resolution (Shortest Edge: 288px) — This proved to be a critical bottleneck. At 288px, dense constraint matrices compress multiple variables into sub-pixel representations. This explains why the model struggled with density and exact variable counting even after fine-tuning.
3. Learning rate — Rates around $1e-4$ were optimal. Rates that were too high caused the model to collapse, outputting repetitive structures, while rates too low failed to overwrite the base model's visual priors.

Problem 6: Post-Training Evaluation (30 pts)

After fine-tuning with LoRA, the model was re-evaluated on the same 30 held-out test images. Fine-tuning produced large improvements in interpreting domain-specific categorical encodings, but exact numerical reasoning remained a persistent challenge:

1. Variable Types (Binary/Integer): The most dramatic improvement. Binary accuracy jumped from 6.7 % $\rightarrow$ 76.7 %, and Integer from 3.3 % $\rightarrow$ 53.3 %. LoRA successfully taught the model to read the top color band.
2. Dimensions (Variables/Constraints): Variables actually decreased from 20.0 % $\rightarrow$ 10.0 %, while Constraints improved from 0.0 % $\rightarrow$ 26.7 %. The model learned to count rows better but became worse at column counting — see ‘new biases’ below.
3. Density: Remained at 0.0 %. The model completely failed to learn this attribute from low-resolution heatmaps.
4. Objective: Remained at 50.0 %. The model still struggles to distinguish the objective row.

New errors and biases introduced by fine-tuning:

The fine-tuned model developed a strong tendency to output fixed ‘prototypical’ values for dimensions (e.g., Variables: 3, Constraints: 4) regardless of the actual image. This is why variable accuracy actually dropped: the base model occasionally guessed correctly by luck, while the fine-tuned model converged to a narrow set of memorised answers. Similarly, density outputs collapsed to a few values (e.g., 0.0, 0.25, 0.5) that rarely match the categorical ground truth (high/medium/low). These are classic signs of mode collapse on a small training set — the model overfits to the most frequent patterns rather than learning to read each image individually.

Overall, LoRA effectively taught the model where to look for Binary/Integer information, but exact counting and continuous attributes like density likely require higher image resolution or a hybrid neuro-symbolic approach.

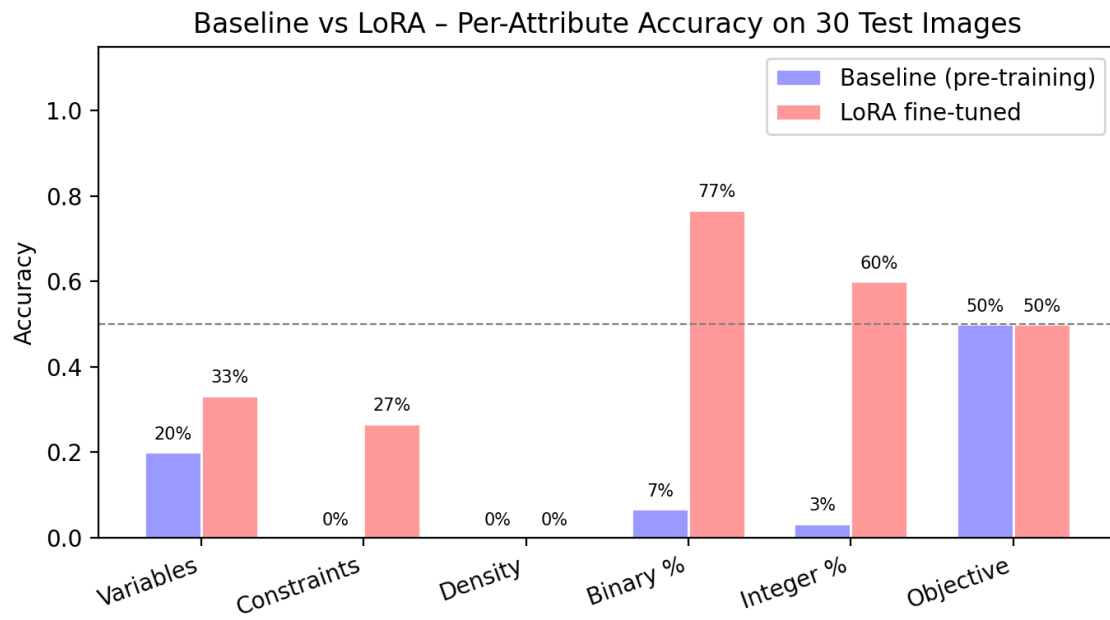


Figure 6.1: Baseline vs LoRA Per-Attribute Accuracy on 30 Test Images

Problem 7: Final Reflection (10 pts)

1. Most interesting concept

The most interesting concept was using heatmap images as a bridge modality for structured numerical data. Transforming a MILP constraint matrix into a raster image that a general VLM can process is elegant modality engineering: instead of building a specialised architecture for tabular inputs, we translate the domain into a modality the VLM already understands. The baseline experiments confirm this works non-trivially — the pre-trained model can estimate density and some dimensions directly from the image.

2. Useful concepts toward the project goal

For S-MIP-Verifier (multimodal MILP consistency checking), LoRA fine-tuning is the planned adaptation strategy — this homework gave hands-on experience tuning rank, alpha, dropout, and target modules, all of which transfer directly. Prompt engineering also showed how structured output format can be enforced via system prompts and few-shot examples, which is essential when designing verification prompts that force the VLM to compare a text description against a visual representation and output a structured verdict.

3. Topics to explore more

I would have liked more depth on evaluation metrics for structured output generation beyond exact match. When the model predicts 'Variables: 3' instead of 'Variables: 2', it is far closer to correct than outputting an unstructured response — yet exact match treats both as fully wrong. Attribute-level F1, normalised edit distance, or token-level partial credit scoring would give a richer picture of how much fine-tuning improves task-specific understanding versus surface format compliance.